

Freight Rate Prediction

Spotter — Machine Learning Engineer assessment

Leutrim Istrefi

1. Summary

48,000 labelled loads run from 1 January to 31 October 2025. The 12,000 loads to price run from **1 November to 31 December** — so this is a two-month-ahead forecast, not a random holdout, and the whole design follows from that.

The model is a gradient-boosted tree on **log dollars per mile**, with the year's upward price drift taken out before fitting and added back at prediction time. Trees cannot extrapolate past the last date they were trained on; handling the drift separately is what makes November and December genuine forecasts rather than a copy of October.

Measured on rolling two-month-ahead holdouts, against labels that were not corrupted:

	Lane median \$/mile	This model	Improvement
MAPE	4.12%	1.80%	2.3× better
MAE	\$98.57	\$43.69	2.3× better
RMSE	\$216.10	\$168.91	1.3× better
R ²	0.975	0.985	—
Mean predicted / actual	0.996	0.997	—

Averaged over three cut-offs (1 Jul, 1 Aug, 1 Sep 2025), each scored on the following 61 days.

For scale: predicting *inside* the training period with a random split — the easiest possible version of the task — gives 1.60% MAPE. The forecast is within 0.2 points of that noise floor, so most of the error that remains is irreducible.

2. The data

	train_test.csv	validation.csv
Loads	48,000	12,000
Dates	2025-01-01 → 2025-10-31	2025-11-01 → 2025-12-31
Label	posted_rate	— (to predict)
Cities	64	72 (8 never seen in training)
Lanes	4,014	4,214 (736 never seen)
Equipment	Dry Van 57%, Reefer 25%, Flatbed 18%	Dry Van 57%, Reefer 25%, Flatbed 18%

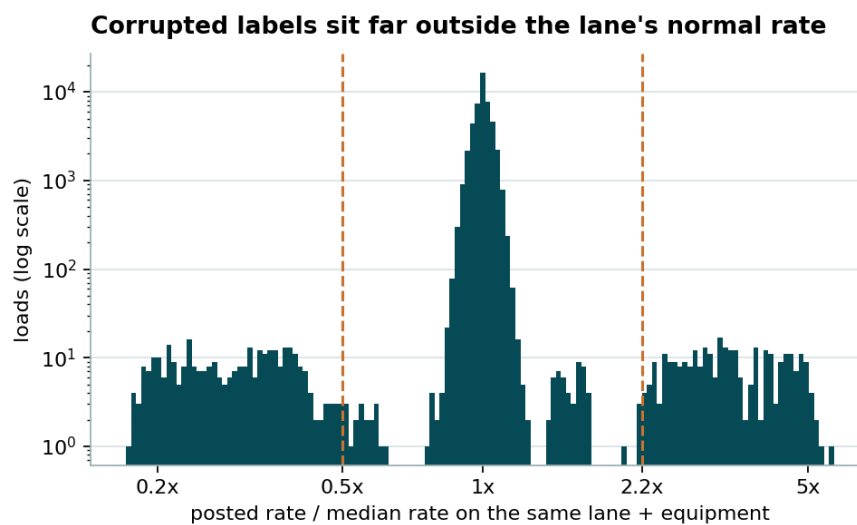
The eight unseen cities and 736 unseen lanes are the reason the model leans on coordinates rather than city names.

3. Data-quality issues found, and what I did

Issue	What it looks like	Treatment
Sign-flipped weights	292 train / 145 validation rows with weight between -47,500 and -8,000 lb; the magnitudes are perfectly normal	abs() — read as a data-entry sign error rather than dropped
Missing values	weight missing on 300 / 165 rows, market_index on 374 / 249	weight left as NaN (the booster routes missing values natively) plus a missing-flag; market_index filled with that day's market level
Corrupted labels	~1.3% of posted_rate values are 3–6× or 0.17–0.4× the normal rate for that lane	Screened out of training in two passes (see below)
Decoy feature	quote_signal looks predictive on raw averages, but the signal disappears once distance, equipment and market are controlled for	Excluded — adding it made holdout MAPE worse (2.63% → 3.15%)
Floored distances	distance never goes below 70 miles, so short city pairs sit at 1.6–9.6× the great-circle distance	Kept, and exposed to the model as a 'circuitry' feature so it can price those hauls separately

The corrupted labels

Dividing each posted rate by the median rate of the same lane and equipment produces three clearly separated groups: a tight core around 1×, a blob of labels at 0.17–0.4×, and a blob at 3–6×. Real price variation does not look like that; these are scrambled labels.

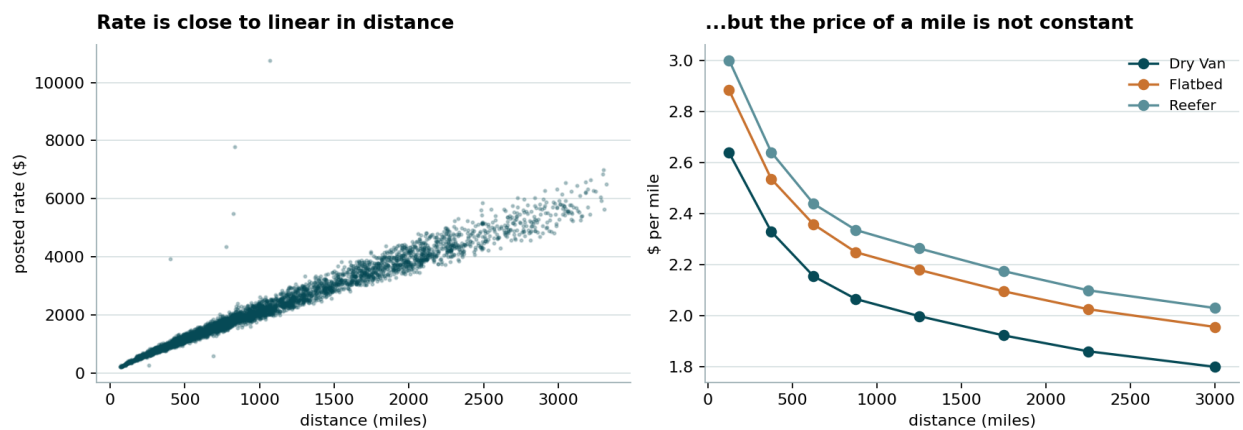


They are screened in two passes. The first uses the lane median above and removes 625 rows (1.30%). The second refits the model, measures each row's residual and drops anything beyond four robust sigmas — this catches the corrupted labels sitting on thin lanes, where the lane median itself was dragged off. Together they remove **1.65% of rows** from training. Dropping both screens costs 1.15 points of MAPE on the two-month-ahead holdouts (1.80% → 2.95%); dropping only the residual pass costs 0.35.

The holdout is deliberately not cleaned in the same way for the headline number — the model is scored on every row. But the corrupted labels are reported separately, because a label that was multiplied by four after the fact is not predictable by any model, and letting 1.3% of rows dominate the metric hides real differences between approaches.

4. What actually moves a rate

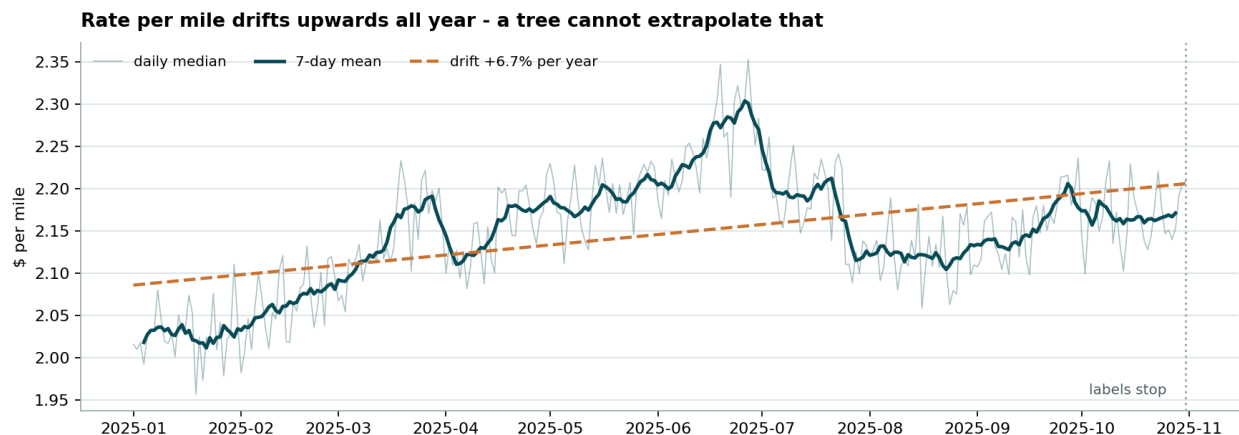
Rate is almost linear in distance ($r = 0.91$), so the model predicts **dollars per mile** and multiplies back. That removes the dominant, trivially-learnable factor and lets the booster concentrate on the interesting part: the price of a mile is not constant.



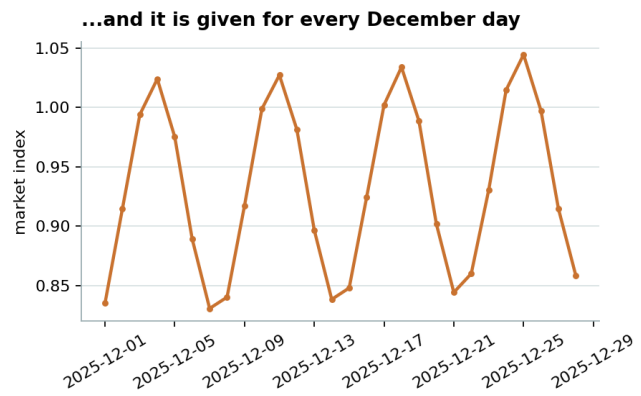
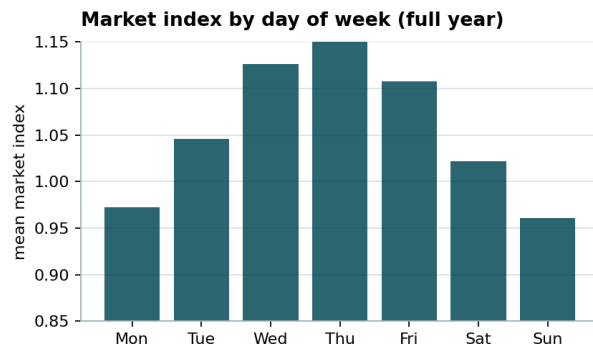
\$/mile falls from ~\$2.80 on short hauls to ~\$1.90 over 2,500 miles, and Reefer carries a ~13% premium over Dry Van at every length.

Three other effects matter, in order of size:

- Market index — a daily level with a clean seven-day cycle (peaks Thursday, troughs Sunday) and a slow seasonal swing. Its daily mean correlates 0.70 with the day-level residual. Within-day variation is noise: the spread within a day is 0.025 against 0.166 across days.
- Drift — rates rose steadily all year, about +6.7% annualised. This is the part a tree cannot extrapolate.
- Weight and geography — heavier loads cost ~8% more per mile from the lightest to the heaviest decile, and origin/destination coordinates carry regional pricing that distance alone misses. Adding coordinates cut holdout MAE by ~18%.



Daily median \$/mile across the labelled period. The dashed line is the drift the model extrapolates into November and December.



The weekly cycle lives in `market_index` — which is published for every December day, so the December chart can follow it.

5. Train/validation split and validation approach

A random K-fold would be wrong here. Loads on the same day, and neighbouring days, share a market level and a price level; a random fold puts some of them in training and some in test, so the model gets to see the future and the score comes out optimistic. The real task extrapolates two months past the last label.

So the split copies the real one exactly: **rolling origin, train on everything before a cut-off, score the next 61 days.**

Cut-off	Training window	Scored on	Holdout loads
1 Jul 2025	1 Jan – 30 Jun	Jul – Aug	9,524
1 Aug 2025	1 Jan – 31 Jul	Aug – Sep	9,429
1 Sep 2025	1 Jan – 31 Aug	Sep – Oct	9,523

Three cut-offs rather than one, because a single two-month window is a small sample of market conditions.

Everything was chosen on these three holdouts and nothing else: the feature set, the label screen, the way the drift is extrapolated, and the booster's size. Three decisions turned on them:

- **Feature selection.** Coordinates helped (−18% MAE); day-of-week, within-day market noise, lane target-encoding and `quote_signal` all hurt or did nothing, and were dropped.
- **Drift extrapolation.** Fitting the drift line on all history under-predicts the recent level by ~1.2%; fitting it on only the last 90 days chases noise. Averaging three fits (all history, last 150 days, last 120 days) was the most accurate and least biased of the options tested, and is what ships.
- **Model capacity.** 31 leaves at 700 iterations; 15 and 63 leaves were both marginally worse. The model is not capacity-limited, it is noise-limited.

The final model is then refitted on all 48,000 labelled loads (Jan–Oct) and applied to the 12,000 validation loads. Its fitted drift is **+0.65% per 30 days**, and the residual scatter around it is 2.2% in log space.

One note on leakage: the daily market level used to fill missing `market_index` values is computed over train *and* validation rows. That column is a published feature, given for every load that has to be priced, so no label information crosses the split.

6. The model

$$\log(\text{posted_rate}) = \log(\text{distance}) + \text{drift}(\text{date}) + \text{booster}(\text{features})$$

Component	Detail
Target	log rate per mile, drift removed. Log space because errors are proportional, not absolute — a \$50 miss on a 200-mile haul is not the same mistake as on a 3,000-mile haul.
Drift	Straight line fitted on the day-level residual, averaged over three look-back windows, extrapolated forward.
Estimator	HistGradientBoostingRegressor — 700 iterations, learning rate 0.05, 31 leaves, min 30 samples per leaf, L2 = 1.0. Handles NaN natively, needs no scaling or one-hot encoding, trains in seconds.
Features (15)	distance, log distance, weight + missing flag, equipment, daily market index, pickup and delivery lat/lon, great-circle distance, circuitry, bearing, route midpoint lat/lon.

What each piece is worth

Each row changes exactly one thing about the shipped model and is re-scored on the same three holdouts:

Variant	MAE	MAPE	RMSE	Δ MAPE
Shipped model	\$43.69	1.80%	\$168.91	—
No residual label screen	\$51.20	2.15%	\$174.06	+0.35
No label screening at all	\$68.47	2.95%	\$186.94	+1.15
Date as a plain feature, no drift term	\$73.73	3.05%	\$189.54	+1.25
No time information at all	\$88.69	3.67%	\$196.70	+1.87
Lane median \$/mile benchmark	\$98.57	4.12%	\$216.10	+2.32

Averaged over the three cut-offs, scored on labels that pass the corruption screen. Reproduce with python -m src.ablation.

Deeper and shallower trees, absolute-error loss, lane target-encoding, day-of-week and quote_signal were all tried and none of them beat the shipped configuration.

7. Results

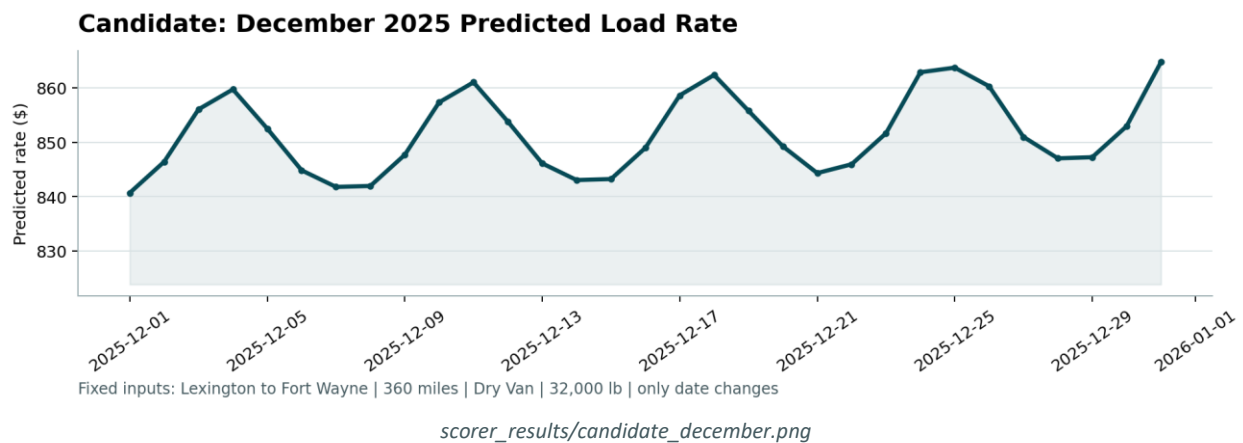
Cut-off	Model	MAE	MAPE	RMSE	R ²
1 Jul 2025	Lane median \$/mile	\$104.26	4.42%	\$202.42	0.978
1 Jul 2025	This model	\$36.83	1.59%	\$137.51	0.990
1 Aug 2025	Lane median \$/mile	\$97.28	4.05%	\$219.81	0.974
1 Aug 2025	This model	\$48.43	1.97%	\$178.93	0.983
1 Sep 2025	Lane median \$/mile	\$94.19	3.91%	\$226.07	0.973
1 Sep 2025	This model	\$45.81	1.83%	\$190.29	0.981
Average	This model	\$43.69	1.80%	\$168.91	0.985

Scored on holdout labels that pass the corruption screen. Over every holdout row, including corrupted labels, the model averages \$92.46 MAE against the benchmark's \$147.70 — the gap between the two columns is the ~1.3% of labels that no model can predict.

The model is very slightly conservative (mean predicted / actual = 0.997). That residual bias is a consequence of extrapolating the drift; damping it further traded away more accuracy than it bought.

8. December chart

The chart below is the file produced by the provided `score.py` from the completed `data/december_chart_inputs.csv` — one Lexington → Fort Wayne load, 360 miles, Dry Van, 32,000 lb, on each of the 31 days of December. Only the date changes.



Two things show up, and both are intended:

- **The weekly cycle.** Rates peak mid-week and bottom out at the weekend, a swing of about \$20 (2.4%). It comes from `market_index`, which is published for every December day, so the model is reading a real given input rather than inventing a pattern.
- **The upward drift.** The month opens around \$841 and closes around \$865. That is the year-long drift, extrapolated — the same mechanism that was validated on the rolling holdouts.

The December rows ship without coordinates or a market index, so both are reconstructed: coordinates from the city table shared by both data sets, market index from the daily level published in `validation.csv`. The level, ~\$2.36 per mile, is consistent with this lane's history — the same lane averaged \$2.19 per mile in January, and eleven months of drift accounts for the difference.

9. Limits and what I would do next

- Ten months of history means the drift can only be read as a straight line. A second year of data would separate trend from seasonality, and December — a month with holiday capacity effects that this data set cannot show — is exactly where that matters.
- The model returns a point estimate. For pricing decisions a quantile version (10th/50th/90th) would be more useful than a single number, and costs almost nothing to add.
- Corrupted labels are screened, not explained. In production the right move is to trace where the 1.3% comes from rather than filter it on every retrain.
- The error left is close to the noise floor of this data, so further gains will come from new information — carrier capacity, fuel, lane-level seasonality — rather than from a bigger model.